



Security Assessment Report

StockApp

Software and Database Security

Group: Masters of Computer Science, Software Engineering
Class of: Fall 2025
Prepared on: 5 April 2026
Authors: Example Group

Contents

Synthesis	1
SQL Injection	2
Command Injection via Ping	4
Plaintext Credentials in config.ini	6
No Brute Force Protection	8

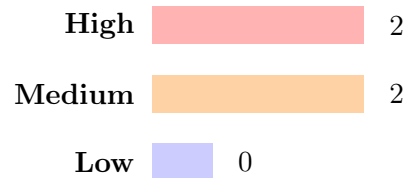
Synthesis

This report presents the results of a security assessment performed on StockApp. The review focused on authentication, input handling, command execution, and local secret storage. This starter version includes four sample findings in order to provide a usable structure for the final report while leaving room for additional vulnerabilities, screenshots, and customer-specific details.

Two of the retained findings are high risk because they allow user-controlled input to alter back-end behavior in a way that can disclose sensitive information or execute unintended commands. The remaining two findings are medium risk because they weaken the protection of credentials and authentication workflows. Even where exploitation requires local access or valid credentials, the resulting exposure remains significant because the application handles account data and interacts with a database.

Priority remediation should focus on removing unsafe SQL and command construction first. After those issues are addressed, secrets should no longer be stored in plaintext configuration files and the login process should be protected against repeated guessing attempts. The final version of the report can extend this synthesis with the full number of findings, exact risk distribution, and any business-specific impact observed during the audit.

Risk level	Number of findings
High	2
Medium	2
Low	0



SQL Injection

Common Weakness Enumeration (CWE) ID

CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

CVSS 3.1 Base Score Metrics

Vector: AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

Score: 7.1

Risk level: High

AV	AC	PR	UI	S	C	I	A
L	L	L	N	U	H	H	N

Description

The application builds a SQL query with user-controlled input instead of separating code from data. A user who reaches the vulnerable search feature can modify the query and retrieve records that should not be exposed. The business impact is unauthorized disclosure of account information and loss of trust in the integrity of the application data.

Exploitation

The vulnerable feature is reached after a normal login to the application.

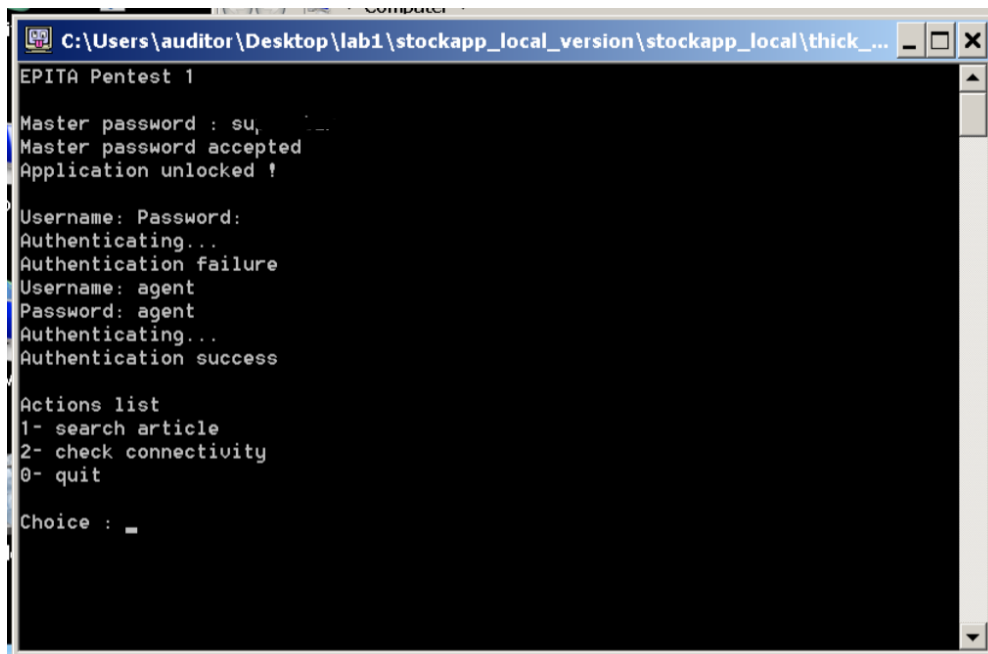
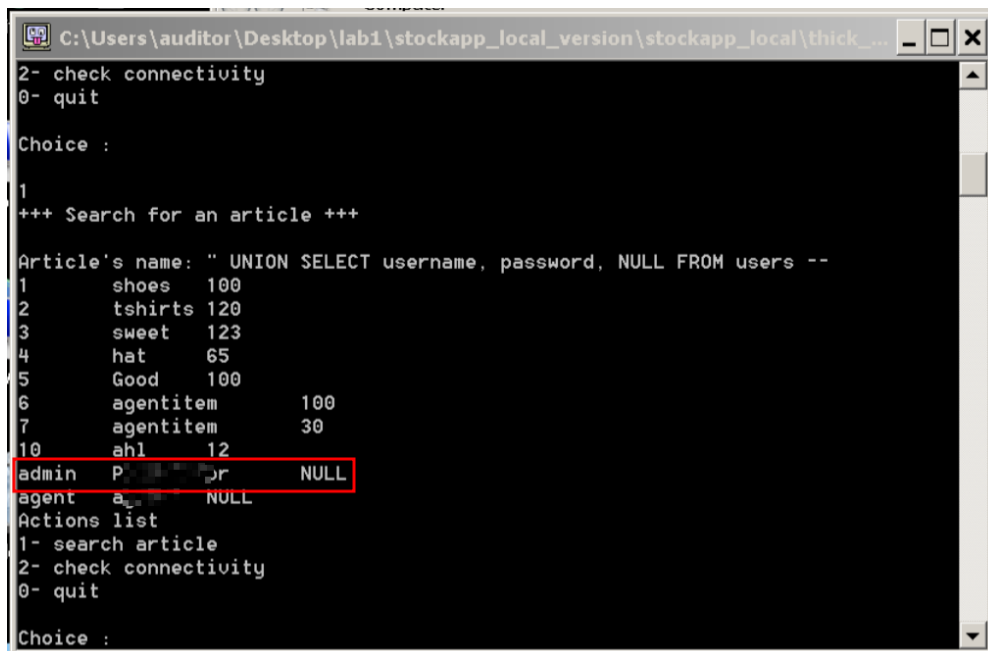


Figure 1: Application menu displayed after a successful login, showing that the article search feature is reachable by a standard user.

Once the search field is available, a crafted payload is entered in place of a normal article name.

A payload such as " UNION SELECT username, password, NULL FROM users; - can force the application to return credential data from the users table.



```
C:\Users\auditor\Desktop\lab1\stockapp_local_version\stockapp_local\thick_...
2- check connectivity
0- quit
Choice :
1
+++ Search for an article +++
Article's name: " UNION SELECT username, password, NULL FROM users --
1 shoes 100
2 tshirts 120
3 sweet 123
4 hat 65
5 Good 100
6 agentitem 100
7 agentitem 30
10 ahl 12
admin P...or NULL
agent a... NULL
Actions list
1- search article
2- check connectivity
0- quit
Choice :
```

Figure 2: Injected SQL payload entered in the article-name field, followed by the returned result containing rows that should not be disclosed.

If usernames and passwords are displayed in the result set, the query has been altered successfully and the vulnerability is confirmed.

Remediation

All database interactions should use parameterized queries or prepared statements. Input validation should reject unexpected values before they reach the SQL layer, and the database account used by the application should be restricted to the minimum privileges required by the feature. Reference: OWASP SQL Injection Prevention Cheat Sheet.

Command Injection via Ping

Common Weakness Enumeration (CWE) ID

CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

CVSS 3.1 Base Score Metrics

Vector: AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Score: 7.8

Risk level: High

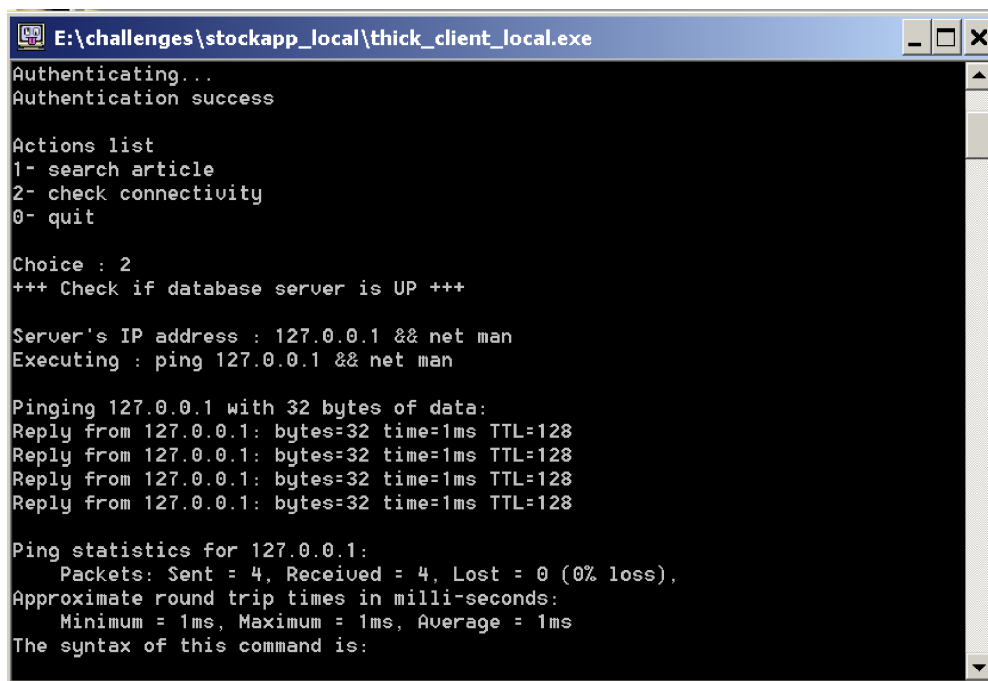
AV	AC	PR	UI	S	C	I	A
L	L	L	N	U	H	H	H

Description

The connectivity test passes user input to an operating system command without safely neutralizing shell metacharacters. A user can therefore append additional commands and execute them in the context of the application. This can expose host-level information or allow unintended actions on the local machine.

Exploitation

The connectivity option is opened from the application menu.



```
E:\challenges\stockapp_local\thick_client_local.exe
Authenticating...
Authentication success

Actions list
1- search article
2- check connectivity
0- quit

Choice : 2
+++ Check if database server is UP +++

Server's IP address : 127.0.0.1 && net man
Executing : ping 127.0.0.1 && net man

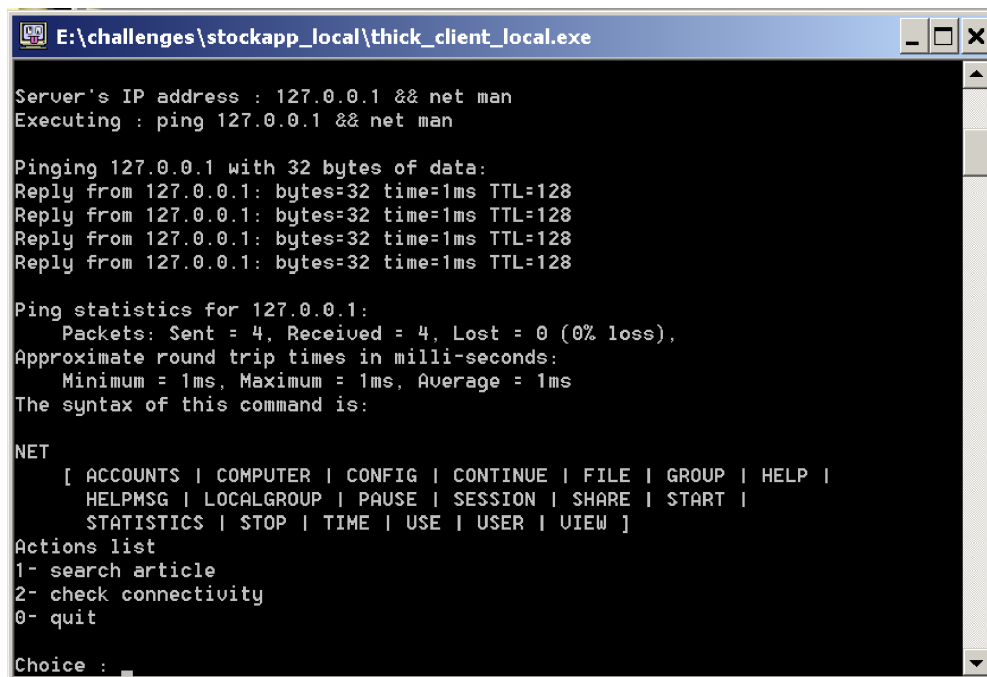
Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128

Ping statistics for 127.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 1ms, Average = 1ms
The syntax of this command is:
```

Figure 3: Connectivity-check screen or console prompt before the host value is entered.

Instead of a normal IP address or hostname, the user enters a chained value such as 127.0.0.1

`&& net user`. If the application interprets the second command and prints its output, the ping feature is vulnerable to command injection.



```
E:\challenges\stockapp_local\thick_client_local.exe

Server's IP address : 127.0.0.1 && net man
Executing : ping 127.0.0.1 && net man

Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128
Reply from 127.0.0.1: bytes=32 time=1ms TTL=128

Ping statistics for 127.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 1ms, Average = 1ms
The syntax of this command is:

NET
[ ACCOUNTS | COMPUTER | CONFIG | CONTINUE | FILE | GROUP | HELP |
  HELPMMSG | LOCALGROUP | PAUSE | SESSION | SHARE | START |
  STATISTICS | STOP | TIME | USE | USER | VIEW ]
Actions list
1- search article
2- check connectivity
0- quit
Choice : 
```

Figure 4: Chained command entered into the host field and the resulting system output produced by the injected command.

The presence of local command output confirms that the application is executing shell content that should have been treated as plain input.

Remediation

The feature should avoid shell execution entirely. If a system call remains necessary, the application must pass the destination as a single validated argument without invoking a command interpreter. The host field should be allowlisted, and the process should run with minimal privileges. Reference: OWASP OS Command Injection Defense Cheat Sheet.

Plaintext Credentials in config.ini

Common Weakness Enumeration (CWE) ID

CWE-312: Cleartext Storage of Sensitive Information

CVSS 3.1 Base Score Metrics

Vector: AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N

Score: 6.4

Risk level: Medium

AV	AC	PR	UI	S	C	I	A
L	L	N	N	U	H	L	N

Description

The application stores sensitive credentials in a local configuration file in plaintext form. Any user with access to the installation directory can read these values and reuse them directly against the associated service. The business impact is exposure of backend access outside the intended application workflow.

Exploitation

The application configuration file is opened from the local installation directory.

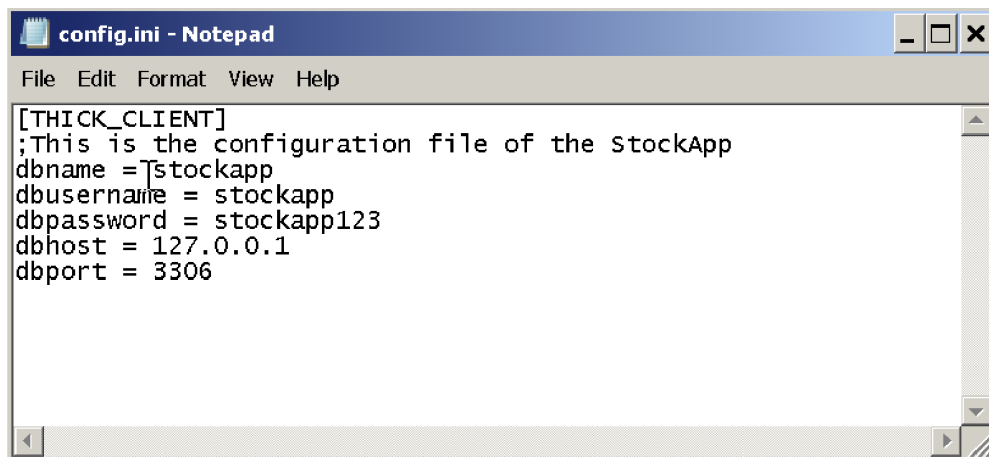


Figure 5: Configuration file opened locally, with database credentials or other sensitive values visible in plaintext.

If usernames, passwords, or connection strings are visible without encryption or further protection, the weakness is confirmed. These values can then be reused directly by an attacker.

Remediation

Sensitive credentials should be removed from flat configuration files and moved to a protected secret store provided by the operating system or deployment environment. Existing secrets

should be rotated and file permissions should be reduced to the minimum account that requires access. Reference: OWASP Secrets Management Cheat Sheet.

No Brute Force Protection

Common Weakness Enumeration (CWE) ID

CWE-307: Improper Restriction of Excessive Authentication Attempts

CVSS 3.1 Base Score Metrics

Vector: AV:L/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N

Score: 5.3

Risk level: Medium

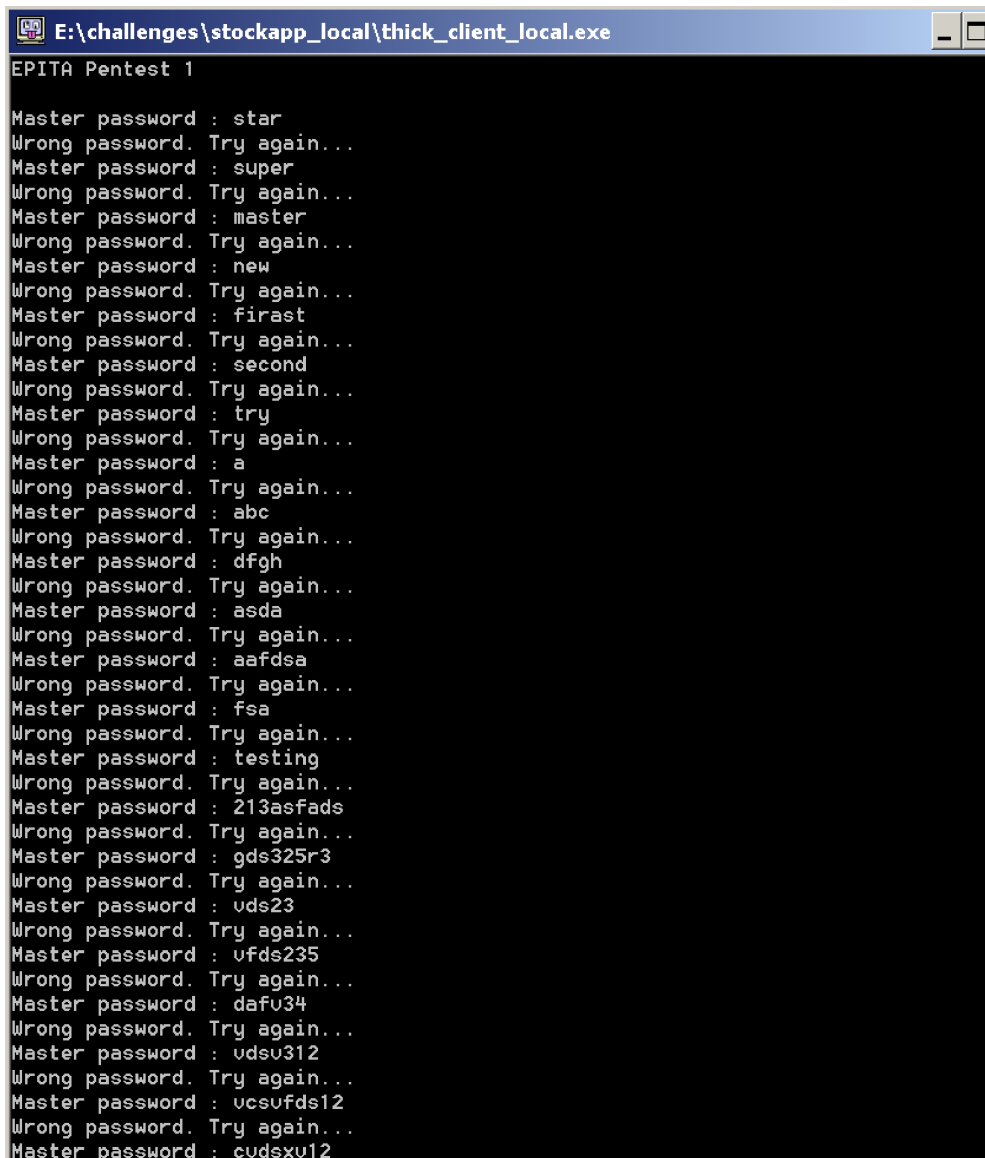
AV	AC	PR	UI	S	C	I	A
L	L	N	N	U	L	L	N

Description

The authentication workflow accepts repeated password attempts without logout, throttling, or meaningful delay. This lowers the cost of password guessing and makes automated attacks practical against weak or reused credentials.

Exploitation

The same account is tested repeatedly with invalid passwords through the login prompt.



```
E:\challenges\stockapp_local\thick_client_local.exe
EPITA Pentest 1

Master password : star
Wrong password. Try again...
Master password : super
Wrong password. Try again...
Master password : master
Wrong password. Try again...
Master password : new
Wrong password. Try again...
Master password : firast
Wrong password. Try again...
Master password : second
Wrong password. Try again...
Master password : try
Wrong password. Try again...
Master password : a
Wrong password. Try again...
Master password : abc
Wrong password. Try again...
Master password : dfgh
Wrong password. Try again...
Master password : asda
Wrong password. Try again...
Master password : aafdsa
Wrong password. Try again...
Master password : fsa
Wrong password. Try again...
Master password : testing
Wrong password. Try again...
Master password : 213asfads
Wrong password. Try again...
Master password : gds325r3
Wrong password. Try again...
Master password : vds23
Wrong password. Try again...
Master password : vdfs235
Wrong password. Try again...
Master password : dafU34
Wrong password. Try again...
Master password : vdsu312
Wrong password. Try again...
Master password : vcsufds12
Wrong password. Try again...
Master password : cudsxu12
```

Figure 6: Repeated failed login attempts for the same account, with no lockout or delay applied between attempts.

If the application continues to accept new attempts immediately and does not introduce a lockout, delay, or challenge step, the absence of brute force protection is confirmed.

Remediation

Authentication attempts should be rate-limited and logged. After a defined number of failures, the application should apply progressive delays, lockout controls, or an additional verification step. Strong password policy enforcement should complement these controls. Reference: OWASP Authentication Cheat Sheet.